Author

HEMLATA MAHAWAR
23F1001797
23f1001797@ds.study.iitm.ac.in

I am currently a student at the diploma level. My technical skills include web development, python, Flask, SQL, Vue.

Description

This multi-user web application allows users to reserve and vacate 4-wheeler parking spots across multiple parking lots, while the admin manages lots, spots, and user data. The app features role-based access control, data visualization, and backend scheduling jobs for reminders, reports, and data exports.
I did not use any LLM/AI to make this project.

Technologies used

BACKEND:

1. Flask: The core web framework used to build the backend of the application, handling routes, requests and responses.
2. Flask-Security-Too: used for session protection.
3. Flask-SQLAlchemy: Used for database integration with SQLite, database management and ORM.
4. SQLite: Database used to store users, parking lots, spots, and booking details.
5. Celery – For asynchronous and scheduled tasks.
6. Redis – Used as a broker and caching layer.

FRONTEND:

7. Jinja2: Used for HTML rendering.
8. VueJS – UI framework for dynamic and component-based front end.
9. Chart.js: For generating charts and graphs in the Admin and User dashboards.
10. Bootstrap, HTML5, CSS: For creating a responsive and user-friendly front-end interface.

Schema Design

ER DIAGRAM

TABLES AND RELATIONSHIPS

1. User table:

   stores user details (including username, password, email, fs_uniquifier, active status, user id.) has one to many relationship with reservation and many to many relationship with role table.

2. ParkingLot table:
   stores lot details (including lot id, prime location name, price, address, pincode, lot capacity, spots_count, created time.) has one to many relationship with parking spot table.
3. ParkingSpot table:
   stores spot details (including spot id, lot id, spot availability status) has one to many relationship with the reservation table.
4. Reservation table:
   stores spot reservation details (including reservation id, spot id, user id, vehicle no, parking timestamp, leaving timestamp, payment status, parking cost, duration.) has many to one relationship with both user and parking spot table.
5. UserRoles table:
   it is an associated table to store user id and their associated role id has many to one relationship with both role and user table.

API Design

All API endpoints are grouped under four main categories: `parking_lot`, `parking_spot`, `users`, and `reservation`. Token-based authentication is required for protected routes (`Authentication-Token` header).

Parking Lot APIs

- GET `/api/parking_lot/get` – Fetch all parking lots (admin only).
- GET `/api/parking_lot/get/{lot_id}` – Fetch a specific parking lot by its `lot_id`.
- POST `/api/parking_lot/create` – Create a new parking lot; parking spots are created automatically based on `capacity`.
- PUT `/api/parking_lot/update/{lot_id}` – Update lot details (note: capacity cannot be changed).
- DELETE `/api/parking_lot/delete/{lot_id}` – Delete a parking lot only if all its spots are available.

Parking Spot APIs

- POST `/api/parking_spot/create/{lot_id}` – Create a new spot for a given lot; only if current `spots_count` < `capacity`.
- DELETE `/api/parking_spot/delete/{spot_id}` – Delete a parking spot if it is currently unoccupied.

User APIs

- GET `/api/user/get` – Get all registered users (admin only).
- GET `/api/user/get/{user_id}` – Get details of a specific user.
- PUT `/api/user/update/{user_id}` – Update user details like email, username, or password.

Reservation API

- POST `/api/reserve_slot/{user_id}/create/{spot_id}` – Create a reservation for a specific user and parking spot. Requires vehicle number and expected leaving time in the request body.

Link for YAML file: Click to open the file

Architecture and Features

Here is the folder structure: main folder contains app.py file, requirement.txt, README.md. Then it contains,

1. Application folder: contains routes.py containing all the controllers, models.py, database.py, config.py, utils.py.
2. Templates folder: contains all jinja2 HTML templates.
3. Static folder: contains style.css and images folder.
4. Instance folder: contains database sqlite file.

Core Features

Admin

- Login via pre-configured superuser (no registration).
- Create, edit, delete parking lots.
- Define the number of spots in a parking lot (auto-created).
- View parking status (occupied/available), user data, and reservation records.
- View dashboard with charts (e.g., most used lots).
- Search functionality across users, lots, reservations, etc.

User

- Register/Login
- Book a parking spot in the selected lot (first-available spot assigned automatically).

- Release reservation.
- View personal parking summary and activity.
- Edit profile: change username, email, password.
- Export parking history as CSV (Async job).

Scheduled & Background Jobs (with Celery + Redis)

1. Daily Reminder Notification

- Runs every evening.
- Sends reminder (via email or Google Chat webhook) to:
    - Users who haven't made a reservation recently.
    - Admins who haven’t created any lots.

2. Monthly User Activity Report

- Runs on the 1st of every month.
- HTML report sent via email including:
    - Total bookings.
    - Most used lot.
    - Total cost spent on parking.

3. CSV Export (User-triggered)

- Async job triggered from user dashboard.
- Prepares downloadable CSV containing user’s parking history.

Performance & Caching

- Redis-based caching for repeated admin/user dashboard data.
- Cache expiry used to maintain freshness.
- Optimized API performance with pagination and filtered queries.

ADDITIONAL FEATURES:

1. Admin has search functionality to search spot/parking lot/users/reservation details by id/name/active status/ etc.
2. API resources are created for parking lots to get, post, put, delete.
3. APIs are created using flask_restful.
4. I have used Chart JS to create charts (pie, bar etc.).
5. I have implemented fronted validation using HTML5 form and backend validation within flask routes.
6. I have done styling using Bootstrap and Inline CSS.

7. I have used flask_security_too to implement authorization and role based access control.
8. I have also added a profile page for users and admin, users and admin can also edit their password, email and username from there.